> **NOTE** You can use `'use server'` in any server component file within the app directory. You can also use it on a function level. Add `'use server'` at the beginning of an async function body to mark only that function as a server action. Always utilize `'use server'` in code that must run on the server, since Next.js will ensure it's only included in the server bundle.

With this in place, we can now remove the API route handler that we had defined `(chat)/api/route.js` since we won't be needing it anymore. Any API calls will be handled by the server actions directly in our components.

That completes the backend part of the implementation. The next section deals with the changes to the UI so that we can call the server actions directly.

4.1.3 Updating the UI to use server actions

Our page.js component, which renders the conversational UI, needs to call the provided `continueConversation` function, passing the message history. We don't have to completely rewrite the existing code. We only need to define a form handler that will call this function and update the client state.

For example, if a user types "What's the weather like today?" our form handler will

1. Capture this input.
2. Add it to the conversation history.
3. Call the `continueConversation` function with the updated history.
4. Receive and display the AI's response, which might be something like "I'm sorry, I don't have access to real-time weather data. However, I can suggest checking a reliable weather website or app for the most up-to-date information."

This process is visualized in figure 4.5.

The following listing showcases the important parts of our form handler implementation.

Listing 4.2 Form handler implementation for chat messages

```
'use client';
// import declarations here
export const dynamic = 'force-dynamic';
export const maxDuration = 30;

const Chat = () => {
  // ... (state and ref declarations)
// const { messages, input, handleInputChange, handleSubmit,
isLoading } = useChat({ api: '/api' });

  const handleOnSubmit = async (event) => {.
    event.preventDefault();
    const value = input.trim();
    setInput('');
```

These export statements ensure that the page is always dynamically rendered and has a maximum execution time of 30 seconds, which is important for handling streaming responses.

This commented-out line shows how we previously used the useChat hook. We're replacing this with our custom implementation using server actions.

The handleOnSubmit function is our custom form submission handler that will use the server action.